



VOLODYMYR STETSENKO

Independent Security Researcher

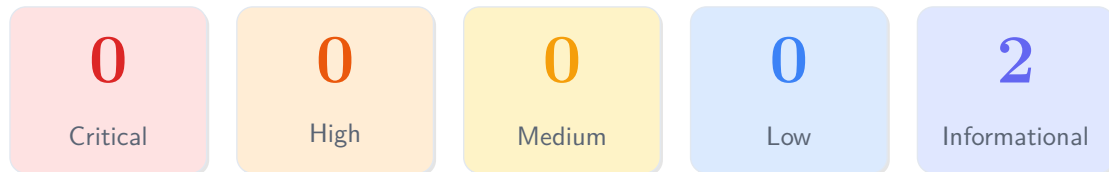
RISC Zero

Ethereum Mainnet Verifier Suite
Security Review

Assessment Period: May 31 – June 02, 2026

Independent Security Research Report

Security Review Overview



Assessment Period: **May 31 – June 02, 2026**
Report Date: **June 02, 2026**
Report Version: **1.0**
Repository Context: **3aa1378 (main)**
Deployment Sources: **Groth16 v3.0.0; v2.2.0 snapshot; SetVerifier v0.9.0**
Solidity Version: **0.8.26 (via_ir, optimizer 10,000)**
Assessment Methods: **Manual Review, Live-Bytecode, Mainnet-Fork Tests**

Contents

1	About Volodymyr Stetsenko	3
2	Disclaimer	4
3	Risk Classification	5
4	About Protocol	5
5	Executive Summary	6
5.1	Audit Overview	6
5.2	Scope	6
5.2.1	Mainnet Footprint (Routable)	7
5.3	Findings Summary	7
6	Threat Model and Security Properties	8
6.1	Adversarial Surfaces Considered	8
6.2	Security Properties Verified	9
6.3	Trust Assumptions	9
7	Methodology	10
7.1	Static source review	10
7.2	Live-mainnet bytecode reconciliation	10
7.3	Prior-art sweep	10
7.4	Live-fork harness	10
7.5	Tools and evidence boundary	10
8	Rust ↔ Solidity Encoding Cross-Check	10
8.1	SetVerifier journal encoding	11
8.2	Merkle leaf vs internal-node domain separation	11
8.3	snarkJS Groth16 verifier hardening	11
9	Live Bytecode Reconciliation	11
10	Live-Fork Probe Suite	12
10.1	Field-edge gate (test 04)	12
10.2	Empirical correction to L-2 (tests 05 and 08)	12
11	Prior-Art Sweep	13

12 Findings	14
12.1 Informational	14
12.1.1 [L-1] v2.2.0 tagged source mismatch with deployed control root	14
12.1.2 [L-2] Groth16 sign-flip seal malleability ($-A, -B, C$) (snarkJS #383 class)	14
13 Engagement Outcome	15
13.1 Observations	16
13.2 Recommendations Beyond the Findings	16
13.3 Reproducibility Notes	16
13.4 Submission Decision	16
13.5 References	16

1. About Volodymyr Stetsenko

Volodymyr Stetsenko is an independent security researcher specialising in the review of EVM-based smart contracts.

- **Areas of practice.** Manual source review, EVM contract testing, deployed-bytecode reconciliation, provenance checks, and verifier-interface security review.
- **Method.** Review of source code against protocol specifications, examination of cryptographic and operational assumptions, and adversarial analysis of logic, encoding, dispatch, and integration boundaries.
- **Reproducibility.** Engagements are documented with the artefacts produced during the review, including fork-test harnesses, command transcripts, bytecode snapshots, and commit-provenance notes where scope permits.

A security review identifies known classes of vulnerability under the constraints of the engagement and does not constitute a guarantee of correctness. Findings reflect the reviewed deployments and source revisions described in the scope section; subsequent changes are not covered by this report.

2. Disclaimer

Important Notice

Volodymyr Stetsenko makes all reasonable efforts to identify vulnerabilities within the reviewed code during the specified period, but holds no liability for the findings presented in this document. A security audit does not constitute an endorsement of the underlying business, product, or protocol.

A smart contract security review **cannot verify the complete absence of vulnerabilities**. This is a time, resource, and expertise-bound effort. **No guarantee of 100% security** is provided, regardless of whether any issues were identified.

The audit was **time-boxed** and focused solely on the security aspects of the Solidity implementation within the defined scope. This report is provided **“AS IS”** without warranty of any kind.

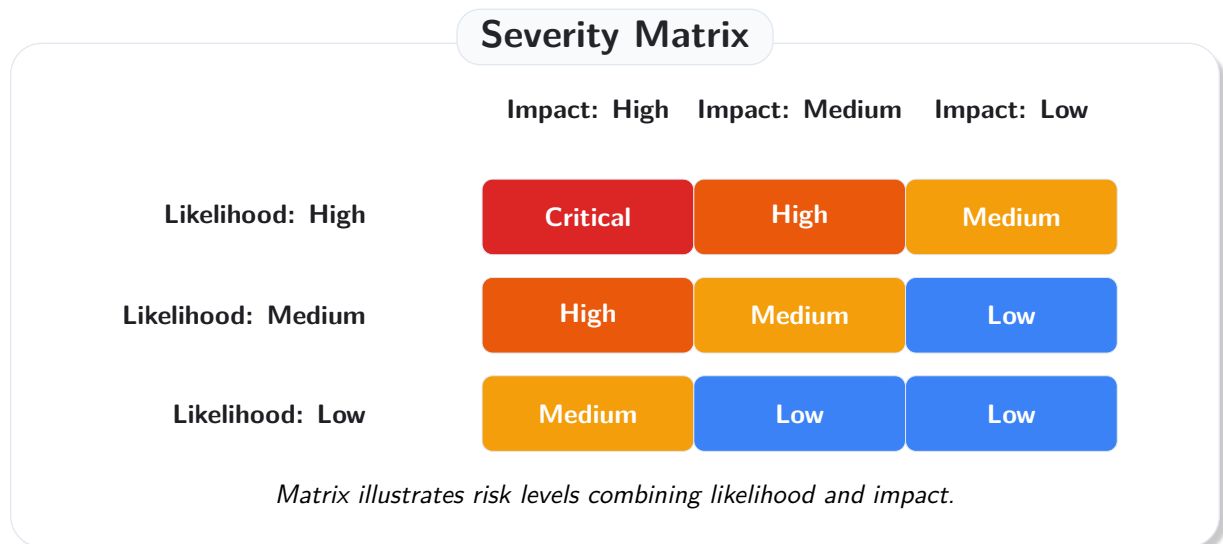
Recommended post-audit measures: subsequent independent reviews, public bug-bounty programs, on-chain monitoring, and formal verification of critical invariants.

Scope Limitations

Unless explicitly stated, the following areas are **outside the scope** of this security assessment:

- RISC Zero zkVM circuits and trusted-setup ceremony
- Off-chain prover SDKs and host bindings
- Set-builder guest program logic
- Economic, governance, and tokenomic analysis
- Frontend / RPC infrastructure
- Third-party integrations consuming the verifier
- Stopped or unroutable historical deployments
- Future code changes and chain migrations

3. Risk Classification



- **High** — results in a significant loss of protocol assets or severely impacts a large user group, or yields invalid claims that successfully verify.
 - **Medium** — results in a moderate loss of protocol assets, partial soundness erosion, or moderately impacts users.
 - **Low** — results in minor asset loss, representation-level malleability, or affects a small group of users.
-
- **High** — attack is feasible under reasonable assumptions on-chain, with relatively low cost versus potential loss.
 - **Medium** — attack is possible under specific conditions and incentives.
 - **Low** — attack requires unlikely assumptions, high cost, or offers little incentive.

4. About Protocol

RISC Zero is a general-purpose zero-knowledge virtual machine (zkVM). The on-chain verifier suite is the trust boundary that consumes Groth16 proofs of zkVM execution and exposes them to Ethereum smart contracts. A consumer contract calls `verifier.verify(seal, imageId, journalDigest)` and the suite resolves the call through three operational layers:

1. `RiscZeroVerifierRouter` — versioned dispatch by 4-byte selector. Reads `seal[0:4]`, looks up the implementation in a tombstone-protected mapping, and forwards the call. Owner-controlled add / remove of verifier versions, with an immutable timelock-controller as the deployment owner.

2. `RiscZeroVerifierEmergencyStop` — a per-verifier OpenZeppelin `Pausable` wrapper. Two pause paths are exposed: an owner-triggered pause, and a permissionless *proof-of-exploit* pause that engages when an inner verifier accepts a receipt with `claimDigest == 0`. The latter is structured as a circuit breaker: engaging it requires an executed soundness violation, not a heuristic signal.
3. `RiscZeroSetVerifier` — recursive aggregation via Merkle Mountain Range commitment. Verifies a single Groth16 “root” proof of the aggregator guest, then offers cheap per-receipt verification via a Merkle inclusion proof. The deployed `VERIFIER` immutable points at the router itself, so the inner Groth16 check re-enters the router for selector dispatch.

The Solidity wrapper contains no circuit logic. All zero-knowledge soundness depends on the Rust prover, the trusted-setup ceremony, and the snarkJS-generated Groth16 verifier; the in-scope contracts are responsible only for digest encoding, dispatch, and operational safety.

5. Executive Summary

This report documents an independent, time-boxed security review of the RISC Zero on-chain verifier suite as deployed on Ethereum mainnet. The engagement combined static source review, reconstruction of public immutables from deployed bytecode for every routable verifier, a Rust-to-Solidity encoding equivalence check at each digest boundary, a survey of prior art against the published Groth16 and zkSNARK vulnerability literature, and a Foundry mainnet-fork negative-test harness comprising ten cases executed against live router and verifier state.

No findings of Critical, High, Medium, or Low severity were identified. Two informational items are recorded: a provenance discrepancy between the `git tag v2.2.0` source revision and the `CONTROL_ROOT` immutable in the deployed v2.2.0 verifier (**L-1**); and the canonical Groth16 sign-flip representation malleability $(-A, -B, C)$ in the deployed Groth16 verifier template (**L-2**), reproduced on a mainnet fork.

5.1 Audit Overview

Attribute	Details
Protocol	RISC Zero on-chain verifier suite (Ethereum L1)
Auditor	Volodymyr Stetsenko (independent)
Engagement	HackenProof — RISC Zero Blockchain Verifiers (Critical-only bounty)
Solidity Version	0.8.26 (<code>via_ir</code> , optimizer 10,000 runs)
Review Period	May 31 — June 02, 2026
Repository Context	<code>3aa1378</code> (<code>main</code>)
Deployment Sources	Groth16 v3.0.0; v2.2.0 deployment snapshot; SetVerifier v0.9.0

5.2 Scope

The engagement covered the active Ethereum mainnet verifier deployments listed in the deployment manifest. It used `3aa1378` as repository context and reconciled deployed constants back to release tags or deployment snapshots. Off-chain components (the zkVM, prover SDKs, host bindings, the trusted-setup ceremony, and the set-builder guest) were out of scope. Stopped or unroutable historical

deployments were excluded.

Contract	Path	nSLOC
RiscZeroVerifierRouter	src/RiscZeroVerifierRouter.sol	~105
RiscZeroVerifierEmergencyStop	src/RiscZeroVerifierEmergencyStop.sol	~72
RiscZeroGroth16Verifier	src/groth16/RiscZeroGroth16Verifier.sol	~184
Groth16Verifier	src/groth16/Groth16Verifier.sol	~195
RiscZeroSetVerifier	src/RiscZeroSetVerifier.sol	~153
IRiscZeroVerifier	src/IRiscZeroVerifier.sol	~213
StructHash	src/StructHash.sol	~66
Util	src/Util.sol	~69
ControlID (v2.2.0/v3.0.0)	src/groth16/ControlID.sol	~10
Steel & shared utilities	remainder of src/	balance
Total		~ 1,159

Mainnet Footprint (Routable)

Role	Address	Selector
Verifier Router	0x8EaB...D319	—
Groth16Verifier v3.0.0	0x2a09...Fa84D	0x73c457ba
Groth16Verifier v2.2.0	0xafB3...9df9	0xbb001d44
SetVerifier v0.9.0	0x5005...EB85	0x242f9d5b
EmergencyStop wrappers	0x9F99...e696 , 0x1efD...D698 , 0x844D...F782	—

5.3 Findings Summary

Severity	Open	Mitigated	Resolved	Total
Critical	0	0	0	0
High	0	0	0	0
Medium	0	0	0	0
Low	0	0	0	0
Informational	2	0	0	2
Total	2	0	0	2

This table summarises all findings identified during the assessment, categorised by severity and remediation status. At the time of report finalisation, two informational findings remain open. No critical, high, medium, or low-severity issues were identified.

Status definitions:

- **Open:** Issue has not been addressed at the time of reporting.
- **Mitigated:** Risk has been partially reduced through compensating controls.
- **Resolved:** Issue has been fully remediated and verified.

6. Threat Model and Security Properties

The engagement was framed by an explicit threat model and a corresponding set of security properties expected to hold against the deployed verifier suite. Every finding category and every test in the live-fork harness traces back to one or more of the entries below.

6.1 Adversarial Surfaces Considered

ID	Surface considered	Disposition
A	BN254 G2 subgroup attack on <code>pB</code> (<code>Groth16Verifier.verifyProof</code>)	Mitigated by the EIP-197 precompile (post-Berlin gnark-crypto enforces subgroup membership).
B	Encoding drift between Rust and Solidity for <code>ReceiptClaim</code> / <code>SystemState</code> / <code>Output</code>	Verified consistent in the encoding cross-check chapter.
C	<code>splitDigest</code> collision or asymmetry in <code>RiscZeroGroth16Verifier</code>	Bijective on <code>bytes32</code> → (<code>bytes16</code> , <code>bytes16</code>).
D	<code>_encodeJournal</code> divergence from the aggregation-guest journal layout in <code>RiscZeroSetVerifier</code>	Verified byte-equivalent to Rust <code>GuestState::encode</code> .
E	G1 coordinate-bound omission for <code>pA</code> , <code>pC</code> , <code>pB</code> (snarkJS issue #423)	Not reproducible as a malleability path against this verifier (live-fork test 05).
F	Selector collision and route-confusion in <code>RiscZeroVerifierRouter</code>	Tombstone mechanism and <code>onlyOwner</code> mutators close this class.
G	Router operator and timelock-key compromise	Owner is a timelock-controller (3-day delay); admin-abuse classes are out of scope per program rules.
H	Merkle leaf vs. internal-node second-preimage in <code>RiscZeroSetVerifier</code>	Length-disjoint domain separation (40-byte leaf vs. 64-byte internal node).
I	EmergencyStop circuit-breaker abuse via <code>claimDigest == 0</code>	Intended design; engaging the breaker requires an executed soundness violation.
J	Permissionless <code>submitMerkleRoot</code> grieving in <code>RiscZeroSetVerifier</code>	Caller bears the cost of a real Groth16 proof; no cross-user impact.
K	Constant-correctness drift in deployed bytecode (KAVEYJOE-class)	Active-mainnet versions verified clean (bytecode reconciliation chapter).
L	Legacy snarkJS scalar-vs-base-field gate (<code>q</code> vs. <code>r</code> ; issue #479, PR #480)	Fix incorporated in deployed bytecode of v2.2.0 and v3.0.0.
M	Groth16 sign-flip seal malleability ($-A, -B, C$) (snarkJS issue #383 class)	Confirmed on a mainnet fork; representation-only — recorded as L-2.
N	Provenance discrepancy between deployed constants and tagged source	Identified for the v2.2.0 <code>CONTROL_ROOT</code> immutable — recorded as L-1.

6.2 Security Properties Verified

The following properties are stated as the on-chain invariants the deployed suite is expected to maintain. Each was checked by the corresponding evidence track and is reported on as part of this engagement.

Property	Statement	Evidence
Selector integrity	The router dispatches <code>seal[0:4]</code> to the unique configured implementation, and a mutated selector is rejected.	Static review; live-fork test 01.
Claim binding	The verifier accepts a seal only for the <code>(imageId, journalDigest)</code> pair under which it was produced; the pair cannot be substituted post-hoc.	Live-fork tests 03 and 08.
Receipt binding	The on-chain <code>claimDigest</code> reconstructed from <code>(imageId, journalDigest)</code> matches the <code>ReceiptClaim</code> digest produced by the Rust prover.	Encoding cross-check; Groth16 wrapper review.
Digest consistency	Every <code>tagged_struct</code> layout is byte-identical between Rust and Solidity, including <code>u16</code> length encoding and exit-code packing.	Encoding cross-check chapter.
Field-bound enforcement	<code>Groth16Verifier.checkField</code> rejects every public signal $\geq r$ before the BN254 pairing precompile is invoked.	Live-fork test 04 (gas-gate measurement).
Dispatch correctness	Each routable selector resolves to the EmergencyStop wrapper of the intended verifier version, and the selector matches the value baked into the verifier runtime bytecode.	<code>eth_call</code> probing of every routable deployment; live-fork test 00b.
Merkle domain separation	Leaf hashes (40-byte <code>LEAF_TAG</code> value) and internal-node hashes (64-byte <code>min</code> <code>max</code>) are length-disjoint, blocking second-preimage substitution.	Encoding cross-check; SetVerifier review.
Constant correctness	<code>CONTROL_ROOT</code> , <code>BN254_CONTROL_ID</code> , <code>SELECTOR</code> , and <code>VERSION</code> on every routable deployment match the source revision recorded for that deployment.	Bytecode reconciliation chapter.
Operational safety	EmergencyStop pause halts a verifier when <code>claimDigest == 0</code> is observed, and the router <code>owner</code> cannot bypass the pause path.	EmergencyStop static review.

6.3 Trust Assumptions

The following assumptions delimit the boundary of the on-chain review. Each is stated explicitly so that downstream readers can audit the assumption set rather than rely on inference.

- **Trusted setup.** The Groth16 trusted-setup ceremony for the RISC Zero proving system is assumed to have been executed correctly; on-chain scope cannot verify the ceremony.
- **Off-chain prover and zkVM circuits.** The Rust prover, the zkVM circuits, and the set-builder guest are assumed to encode `ReceiptClaim`, `SystemState`, `Output`, and `MMR` byte-for-byte as the on-chain wrapper expects. The Rust-to-Solidity equivalence check in this report bounds this assumption to the digest boundary.
- **Privileged roles.** The router `owner` (timelock-controller) and the EmergencyStop `owner` are assumed honest within the bounds of the bounty program. Admin-key compromise is out of program scope.
- **Ethereum mainnet precompiles.** The post-Berlin EIP-196 / EIP-197 precompiles enforce on-curve and subgroup membership for BN254 inputs.
- **Solidity compiler.** The Solidity compiler 0.8.26 with `via_ir` and 10 000 optimizer runs is assumed to translate the verifier source into bytecode without semantics-altering optimisations. The bytecode-reconciliation chapter probes the deployed runtime directly and therefore does not depend on this assumption for the findings.

7. Methodology

The engagement proceeded along four tracks. Candidate issues were recorded only when supported by source review and at least one reproducible evidence trail.

7.1 Static source review

Marker-by-marker reading of the Solidity verifier sources. Every digest definition was compared against the upstream Rust claim, structural-hash, system-state, and aggregation implementations.

7.2 Live-mainnet bytecode reconciliation

Mainnet `eth_call` probes read every public immutable and route address on the routable deployments. Raw runtime bytecode was archived; a `git log -S` sweep resolved each constant to its source-of-truth commit.

7.3 Prior-art sweep

0xPARC ZK Bug Tracker (27 wild + 8 common-vulnerability categories), the snarkJS issue tracker (#57, #114, #120, #383, #423, #432, #479; PR #480), Beosin and Geometry malleability research, the KAVEYJOE WEB3-555 fix, and the Aztec Plonk “0-bug” literature.

7.4 Live-fork harness

A Foundry mainnet-fork suite of ten probes (`test/AuditForkNegative.t.sol`) executed against the deployed router, the v3.0.0 verifier, and the EmergencyStop wrapper. All ten tests pass; the sign-flip malleability probe is assertion-based and fails if the mutated proof stops verifying the same claim.

7.5 Tools and evidence boundary

The evidence in this report relies on manual review, Foundry fork tests, direct `eth_call` / `cast` probing, bytecode snapshots, and `git` history inspection. No formal-verification, symbolic-execution, or fuzzing-campaign claim is made for this engagement. Those techniques remain valid follow-up options, but they are not presented as evidence here.

8. Rust ↔ Solidity Encoding Cross-Check

The Solidity wrapper consumes digests produced by the Rust prover. A drift in any `tagged_struct` layout between the two would be a Critical bug because it would let a malicious prover bind a different statement than the one that the on-chain verifier reconstructs.

The shared layout is:

```
sha256( tag_digest || down[0] || ... || down[N-1]
        || data_word_0_LE || ... || data_word_K_LE
        || N_LE_u16 )
```

Every tagged struct used by the verifier was checked. Results:

Struct	Tag	Down fields	u16	Verdict
ReceiptClaim	ReceiptClaim	input, pre, post, output	4	match
SystemState	SystemState	merkle root	1	match
Output	Output	journal digest, assumptions digest	2	match

The `ReceiptClaim` exit-code packing requires a brief note. Solidity emits `uint32(byte) << 24`, which serialises as `[byte, 0, 0, 0]`. Rust emits the same four bytes for one-byte values. The Solidity enum and `uint8` user code preclude overflow into the second byte, so the on-chain encoding domain is a strict subset of the off-chain domain. No spoofing path was identified at this boundary.

8.1 SetVerifier journal encoding

Rust (`crates/aggregation/src/lib.rs`):

```
GuestState::encode = self_image_id (32B) || MMR::encode
MMR::encode = bitmap_BE (32B, bit 255 set)
              || peak.digest (32B)
```

Solidity (`RiscZeroSetVerifier._encodeJournal`):

```
1 abi.encodePacked(
2     IMAGE_ID,
3     uint256(1 << 255),
4     root
5 ) // 32 + 32 + 32 = 96 bytes
```

The bitmap word packs as `[0x80, 0x00, ..., 0x00]`, matching the Rust encoding. The full 96-byte journal therefore matches byte-for-byte.

8.2 Merkle leaf vs internal-node domain separation

Rust and Solidity both hash leaves as `LEAF_TAG || value` over **40 bytes**. Internal nodes use OZ Merkle hashing over **64 bytes**. Length-disjointness blocks the classic second-preimage attack.

8.3 snarkJS Groth16 verifier hardening

The deployed snarkJS-generated `Groth16Verifier.sol` for both v2.2.0 and v3.0.0 is byte-identical and incorporates:

- `lt(v,r)` **field check** — correct scalar field, fixed in snarkJS PR #480 / issue #479. Confirmed by gas-gate test (§10, test 04).
- `-A` **negation via** `mod(sub(q, _pA[1]), q)` — relies on the EIP-196/197 precompile to enforce on-curve and subgroup membership. The coordinate-bound omission identified in snarkJS issue #423 is present in the deployed template, but did not produce a malleability path against this verifier in testing (test 05; analysis in §10).
- **G2 subgroup membership of** `_pB` — enforced by the modern Ethereum mainnet pairing precompile (post-Berlin gnark-crypto). Historically a concern (Aztec Plonk “0-bug”), now closed at the precompile layer.

9. Live Bytecode Reconciliation

Every public immutable on each routable deployment was read via `eth_call`, the `splitDigest` reversed, and the underlying `CONTROL_ROOT` reconstructed.

Verifier	Reconstructed <code>CONTROL_ROOT</code>	Origin in repository
v3.0.0	<code>0xa54dc85a...998c1f56</code>	matches the <code>v3.0.0</code> tag
v2.2.0	<code>0xce52bf56...2c5a1529</code>	matches commit <code>4666fad6</code> (deploy-time snapshot); <i>not</i> the later <code>git tag v2.2.0</code> — see L-1

`BN254_CONTROL_ID` is `0x04446e66...4657ac0` on both deployments and identical to every relevant `ControlID.sol` revision. Both selectors (`0x73c457ba`, `0xbb001d44`) match the deployment manifest `contracts/deployment.toml`. `SetVerifier imageInfo` returns `imageId = 0x70909b25...02cc9`, matching the manifest's `set-builder-image-id`. The `RiscZeroSetVerifier.VERIFIER` immutable resolves to the router itself, so a `SetVerifier rootSeal` re-enters the router for the inner Groth16 check — confirmed by direct `cast call`.

10. Live-Fork Probe Suite

The harness `test/AuditForkNegative.t.sol` pins a previously verified v3.0.0 receipt and executes ten fork probes against live mainnet router and verifier state. All ten test cases pass. Most probes are negative-path rejection checks; test 08 is intentionally positive and asserts that the canonical sign-flipped proof verifies the same claim.

#	Probe	Result
00a	Baseline live <code>verify()</code> of v3.0.0 receipt	verifies
00b	Router routes selector <code>0x73c457ba</code> → estop <code>0x9f99...e696</code>	verifies
01	Selector spoofing: mutate <code>seal[0:4]</code>	reverts <code>SelectorMismatch</code>
02	Identity points $a = c = (0, 0)$, $b = ((0, 0), (0, 0))$	reverts <code>VerificationFailed</code>
03	Tampered <code>journalDigest</code>	reverts <code>VerificationFailed</code>
04	Field-edge gate: <code>pubSignal == r</code> vs <code>r-1</code>	gas-gated
05	Out-of-range <code>A.y + q</code> (naive snarkJS #423 vector)	does not verify
06	Control: <code>A.x + q</code> (raw to precompile)	reverts (off-curve)
07	Truncated seal (head bytes only)	reverts
08	Canonical Groth16 sign-flip $(-A, -B, C)$	verifies same claim

10.1 Field-edge gate (test 04)

`Groth16Verifier.checkField` must reject any public signal $\geq r$ before the BN254 pairing precompile is invoked. The gas cost of the assertion provides a direct empirical signature of the gating order:

Public input	Gas observed	Conclusion
<code>pubSignal == r</code>	7,175	rejected at <code>checkField</code> ; pairing not reached
<code>pubSignal == r-1</code>	$\sim 1.04 \times 10^9$	reaches the BN254 pairing precompile

The gas differential of approximately five orders of magnitude is consistent with `lt(v, r)` executing prior to any field-bound operation. The presence of the snarkJS PR #480 fix — which gates against the *scalar* field r rather than the base field q — is therefore confirmed in the deployed bytecode of both v2.2.0 and v3.0.0.

10.2 Empirical correction to L-2 (tests 05 and 08)

The hypothesis associated with snarkJS *issue #423* — that adding q to a G1 coordinate yields a proof that still verifies — does not hold against this verifier. The negation in the deployed template is implemented as `mod(sub(q, A.y), q)`,

where the inner `sub` is 256-bit arithmetic and only the outer `mod` reduces by q . For an out-of-range submission $A'_y = A_y + q$:

$$\text{sub}(q, A_y + q) = 2^{256} - A_y \pmod{2^{256}}, \quad (2^{256} - A_y) \bmod q \neq (q - A_y) \bmod q,$$

because $2^{256} \not\equiv 0 \pmod{q}$. The out-of-range coordinate is accepted syntactically but maps to a *different* (failing) pairing input, so issue #423 is **not** exploitable here as seal malleability.

The malleability that does hold against this verifier is the canonical Groth16 sign-flip variant (snarkJS issue #383 class): for any valid proof (A, B, C) , the bilinear-pairing identity $e(-A, B) = e(A, -B)$ implies that $(-A, -B, C)$ satisfies the same pairing equation. Negating A 's G1 coordinate ($y \mapsto q - y$) and both G2 y -limbs of B yields a `seal` that differs in the encoded byte-string from the original yet verifies the same `(imageId, journalDigest)`. The behaviour was reproduced on a mainnet fork; see L-2.

11. Prior-Art Sweep

Source	Coverage	Result
0xPARC ZK Bug Tracker	27 wild + 8 common categories	none applicable — Solidity wrapper contains no circuit logic; trusted-setup integrity is off-chain
snarkJS #57, #114, #120, #383, #423, #432, #479; PR #480	verifier-template hardening	#479 / #480 (q -vs- r): already fixed (verified in deployed bytecode); #423 ($A_y + q$): not reproducible as a malleability path against this verifier (test 05); #383 sign-flip: confirmed malleable but representation-only (L-2)
Beosin, "Transaction Malleability of Groth16 Proof" (2023)	three constructions	$(-A, -B, C)$ reproduces on the live verifier (test 08); statement-binding intact
Geometry, "Groth16 Malleability"	mitigation patterns	RISC Zero applies upstream snarkJS fix; no further mitigation required at verifier layer
KAVEYJOE WEB3-555 (HackenProof, fixed <code>bd83fe1</code> , Dec 2025)	constant-correctness V5.0	active mainnet versions verify clean per the bytecode reconciliation chapter
Aztec Plonk "0-bug" (Quan)	identity-point / on-curve validation	EIP-197 precompile enforces on-curve and subgroup; not reachable on Ethereum L1 today

12. Findings

12.1 Informational

INFORMATIONAL

[L-1] v2.2.0 tagged source mismatch with deployed control root

Severity: **INFO**Status: **Open**Location: `ControlID.sol`

Class: Provenance / source-tag drift

Description

The `git tag v2.2.0` revision of `contracts/src/groth16/ControlID.sol` carries `CONTROL_ROOT = 0x884389273e128b32...`, which is *not* the value baked into the live `0xafB31f5b...` deployment. The reconstructed live root is `0xce52bf56...2c5a1529`.

A `git log --all -S<value>` sweep against the entire repository history resolves the deployed root to commit `4666fad6`, an earlier development snapshot which was the actual deployment point. This is consistent with the parallel `bitvm/ControlID.sol` present in that commit. The off-chain prover and the on-chain verifier therefore agree on the same value, so the deployment is internally consistent — the issue is purely one of documentation provenance.

Impact

Auditors and integrators reading the `git tag v2.2.0` source will compute a different control root from the value present in the deployed verifier and may incorrectly conclude that a soundness-affecting deviation exists between source and chain. The condition is one of documentation provenance and operational hygiene; it does not affect statement-binding or pairing-equation correctness, since the off-chain prover and the on-chain verifier are configured with the same control root.

Recommended Mitigation

Either retag `v2.2.0` to point at the actual deployment commit `4666fad6`, or add an explicit provenance field per verifier in `contracts/deployment.toml`:

```
[verifiers.groth16_v2_2_0]
address      = "0xafB31f5b70623CDF4b20Ada3f7230916A5A79df9"
deployed-from-commit = "4666fad6"
```

This makes provenance unambiguous for downstream auditors and prevents the same class of confusion on future verifier deployments.

INFORMATIONAL

[L-2] Groth16 sign-flip seal malleability $(-A, -B, C)$ (snarkJS #383 class)

Severity: **INFO**Status: **Open**Location: `Groth16Verifier.sol`

Class: Representation malleability

Description

For any valid Groth16 proof (A, B, C) accepted by the deployed verifier, the variant $(-A, -B, C)$ can be constructed deterministically: the G1 point A is negated ($y \mapsto q - y$) and the G2 point B is negated on both y -limbs. Because the bilinear pairing satisfies $e(-A, B) = e(A, -B)$, this variant satisfies the same

pairing equation as the original and yields a `seal` byte-string distinct from the original that verifies the same `(imageId, journalDigest)`.

The behaviour is a property of the Groth16 proof system rather than of this contract, and corresponds to the snarkJS issue #383 class previously documented by Geometry and Beosin. Statement-binding is preserved: the `(imageId, journalDigest)` pair bound to the verification cannot be altered.

Impact

Downstream contracts that derive a unique identifier for a verification event from `keccak256(seal)` — for example, for replay protection, nullifier tracking, or payment-uniqueness checks — will treat the two equivalent proof encodings as distinct events. Depending on the consumer's logic, this may permit duplicated voucher redemption, repeated reward issuance, or double-counted state transitions at the integrator layer. The verifier itself remains sound; the impact is confined to consumers that anchor uniqueness on the seal byte-string.

Proof of Concept

The behaviour was reproduced on a mainnet fork against the v3.0.0 verifier (test 08 in `test/AuditForkNegative.t.sol`). The naive #423 variant ($A_y + q$) is excluded by test 05; only the canonical sign-flip variant satisfies the pairing equation:

```
1 // (-A, -B, C): G1 negation y -> q - y; G2 negation on both y-limbs.
2 s.a[1]      = Q - s.a[1];
3 s.b[1][0]   = Q - s.b[1][0];
4 s.b[1][1]   = Q - s.b[1][1];
5
6 bytes memory mutated = bytes.concat(bytes4(SELECTOR_V3), abi.encode(
7     s));
8 verifier.verify(mutated, TestReceipt.IMAGE_ID, journalDigest); //
9     succeeds
```

The mutated seal differs in 96 bytes from the original yet verifies the same claim — forge log: `(-A,-B,C) verifies same claim? YES`.

Recommended Mitigation

This is an **integrator-side** concern; enforcing a canonical proof representation at the verifier level is not generally possible without breaking compatibility with all snarkJS-generated provers. The recommended remediation is documentation:

- Add a NatSpec section to `IRiscZeroVerifier` stating that consumers *must* anchor uniqueness on `(imageId, journalDigest)` or on `claimDigest`.
- Explicitly warn that `keccak256(seal)` is **not** a unique identifier of a verification event.
- Optionally publish a small example showing the sign-flip variant so that integrators can build a regression test against the malleable form.

13. Engagement Outcome

The active-mainnet RISC Zero verifier deployments are consistent with their source revisions (with the documentation-provenance exception recorded in L-1), align byte-for-byte with the upstream Rust prover at every encoding boundary examined in this review, and incorporate the verifier-template hardening published by *iden3/snarkjs* (PR #480, scalar-field gate). The two findings recorded (L-1, L-2) are informational and do not affect statement-binding, soundness, or operational safety of the verifier as deployed.

The absence of higher-severity findings is not treated as a gap to be filled by severity inflation. The bounty impact is narrow: an invalid RISC Zero zkVM Groth16 receipt claim must successfully verify on-chain. No path to that impact was identified during this review.

13.1 Observations

- **Compact code base.** 1,159 nSLOC across 11 Solidity files, with no unnecessary indirection between the router, the pause wrapper, and the verifier implementation.
- **Layered operational controls.** Selector-dispatched router with a per-verifier OpenZeppelin `Pausable` EmergencyStop wrapper and a proof-of-exploit pause path triggered by `claimDigest == 0`.
- **Encoding equivalence.** Byte-level agreement between the Rust digest definitions and the Solidity `StructHash` implementation at every `ReceiptClaim`, `SystemState`, and `Output` boundary.
- **Verifier hardening present in deployed bytecode.** The `lt(v, r)` scalar-field check from snarkJS PR #480 is present in both the v2.2.0 and v3.0.0 deployments, confirmed by the field-edge gas-gate measurement.
- **Live-state validation.** Ten of ten negative test cases executed against mainnet-fork state behave as specified, including the field-edge gas-gate measurement and the rejection of the out-of-range coordinate variant.

13.2 Recommendations Beyond the Findings

1. Document in verifier NatSpec that integrators must anchor uniqueness on the claim tuple or `claimDigest`; `keccak256(seal)` is not unique.
2. Add a `deployed-from-commit` field per verifier in the deployment manifest. This removes the provenance ambiguity described in L-1 for future deployments.
3. Maintain the Rust-to-Solidity digest equivalence check as a continuous-integration invariant on every release. It is the principal correctness check at the verifier-wrapper boundary.

13.3 Reproducibility Notes

- Fork harness command:

```
forge test --match-path test/AuditForkNegative.t.sol \
  --fork-url <mainnet-rpc> -vvv
```

Result: 10 passed; 0 failed; 0 skipped. Test 08 asserts the sign-flip outcome and logs:

```
(-A,-B,C) verifies same claim?: YES (malleable)
```

- v2.2.0 provenance check:

```
git show v2.2.0:contracts/src/groth16/ControlID.sol
git show 4666fad6:contracts/src/groth16/ControlID.sol
```

The release tag shows `CONTROL_ROOT = 0x88438927...`. The deployment snapshot shows the `0xce52bf56...` root recorded in L-1.

- Bytecode snapshots used during review are retained under `audit/bytecode/`. They are evidence artefacts only and are not part of the upstream RISC Zero repository.

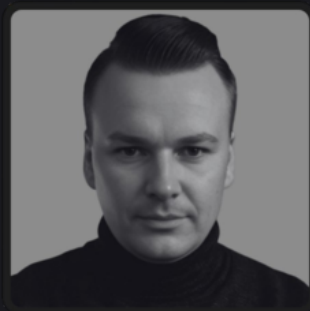
13.4 Submission Decision

The HackenProof program is scoped to Critical severity. Neither L-1 nor L-2 meets that threshold and no bounty submission is made. This report is prepared as independent security research; any external distribution should comply with the program's coordinated-disclosure requirements.

13.5 References

- iden3/snarkjs issues #383, #423, #479; PR #480.
- 0xPARC `zk-bug-tracker` — 27 ZK bugs in the wild and 8 common-vulnerability categories.
- EIP-196 / EIP-197 — BN254 precompile semantics.
- RISC Zero `RELEASE.md` and `contracts/deployment.toml`.

- J.-P. Aumasson, *Security of ZKP projects: same but different*, ZK7.
- Geometry, *Groth16 Malleability*.
- Beosin Research, *Transaction Malleability of Groth16 Proof*, June 2023.
- N. T. M. Quan, *Aztec Plonk Verifier 0-bug*.
- ePrint 2018/187, 2019/586, 2020/811, 2020/1306 — non-malleable Groth16 constructions.
- HackenProof advisory WEB3-555 (KAVEYJOE, December 2025).



Volodymyr Stetsenko

Independent Security Researcher

✉ volodymyrstetsenkoaudit@gmail.com

🌐 [Volodymyr Stetsenko](#)

🐦 [Volodymyr Stetsenko](#)

🔄 [Volodymyr Stetsenko](#)

Independent Security Research Report

© 2026 Volodymyr Stetsenko. All rights reserved.